

Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет
информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра информатики
Дисциплина «Методы защиты информации»

ОТЧЕТ
к лабораторной работе № 1
на тему «Симметричная криптография. Стандарт шифрования ГОСТ
28147-89»

Выполнил

Е. С. Кахновский

Проверил

Е. А. Лещенко

Минск 2024

СОДЕРЖАНИЕ

1 Постановка задачи.....	3
2 Краткие теоретические сведения.....	4
3 Результаты выполнения лабораторной работы.....	5
Выводы	15
Список использованных источников	7
Приложение А (обязательное) Листинг исходного кода	8

1 ПОСТАНОВКА ЗАДАЧИ

Целью выполнения данной лабораторной работы является реализация программных средств шифрования и дешифрования текстовых файлов при помощи стандарта шифрования ГОСТ 28147-89 в режиме простой замены.

2 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

ГОСТ 28147-89 «Системы обработки информации. Защита криптографическая. Алгоритм криптографического преобразования» - устаревший государственный стандарт СССР, описывающий алгоритм симметричного блочного шифрования и режимы его работы. Симметричные криптосистемы – способ шифрования, в котором для шифрования и расшифрования применяется один и тот же криптографический ключ. Блочный шифр – разновидность симметричного шифра, оперирующего группам бит фиксированной длины – блоками, характерный размер которых меняется в пределах 64-256 бит. Если исходный текст меньше размера блока, перед шифрованием его дополняют.

ГОСТ 28147-89 является примером DES-подобных криптосистем, созданных по классической итерационной схеме Фейстеля. DES – алгоритм для симметричного шифрования, разработанный фирмой IBM и утвержденный правительством США в 1977 году как официальный стандарт. Размер блока для DES равен 64 битам. В основе алгоритма лежит сеть Фейстеля с 16 циклами и ключом, имеющим длины 56 бит.

Сеть Фейстеля, или конструкция Фейстеля – один из методов построения блочных шифров. Сеть состоит из ячеек, называемых ячейками Фейстеля. На вход каждой ячейки поступают данные и ключ. На выходе каждой ячейки получают измененные данные и измененный ключ. Все ячейки однотипны, и считается, что сеть представляет собой определенную итерированную структуру. Ключ выбирается в зависимости от алгоритма шифрования или дешифрования и меняется при переходе от одной ячейки к другой. При шифровании и расшифровании выполняются одни и те же операции, отличается только порядок ключей.

Выделяют четыре режима работы ГОСТ 28147-89:

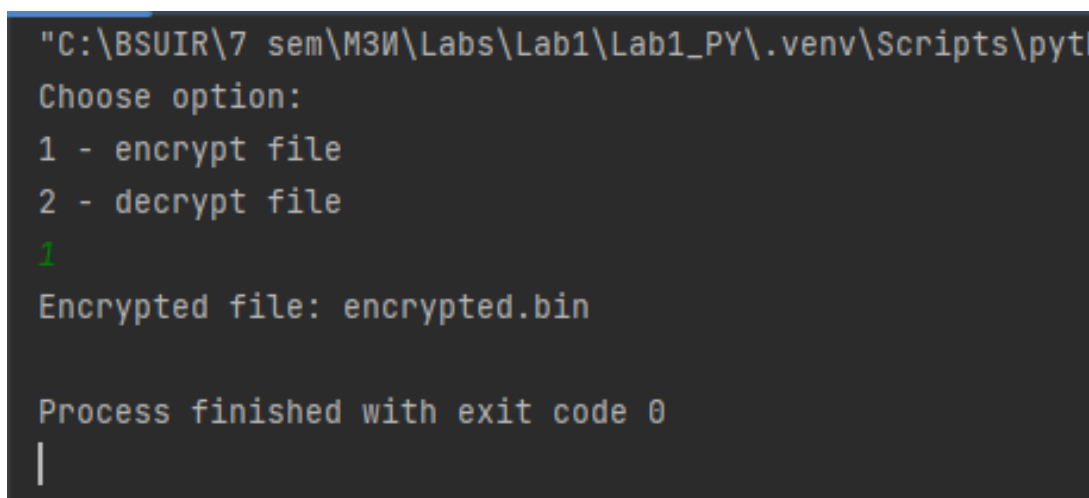
- простой замены;
- гаммирование;
- гаммирование с обратной связью;
- режим выработки имитовставки.

Шифрование по ГОСТ 28147-89 в режиме простой замены — это один из режимов работы блочного симметричного шифра, который описан в российском стандарте ГОСТ 28147-89. Этот режим называют также электронной книгой кодов или режимом простой замены. В этом режиме каждый блок данных шифруется отдельно с использованием одного и того же ключа.

3 РЕЗУЛЬТАТЫ ВЫПОЛНЕНИЯ ЛАБОРАТОРНОЙ РАБОТЫ

В ходе выполнения лабораторной было реализовано программное средство шифрования и дешифрования текстовых файлов при помощи стандарта шифрования ГОСТ 28147-89 в режиме простой замены.

Начальный текст находится в файле input.txt. После шифрования зашифрованная информация помещается в файл encrypted.bin, после чего она снова дешифруется и помещается в файл decrypted.txt. Результат работы программы представлен на рисунке 3.1.



```
"C:\BSUIR\7 sem\МЗИ\Labs\Lab1\Lab1_PY\.venv\Scripts\python.exe"
Choose option:
1 - encrypt file
2 - decrypt file
1
Encrypted file: encrypted.bin

Process finished with exit code 0
|
```

Рисунок 3.1 – Результат работы программы

Таким образом, в ходе данной лабораторной работы было реализовано программное средство шифрования и дешифрования текстовых файлов при помощи стандарта шифрования ГОСТ 28147-89 в режиме простой замены.

ВЫВОДЫ

В ходе данной лабораторной работы было реализовано программное средство шифрования и дешифрования текстовых файлов при помощи стандарта шифрования ГОСТ 28147-89 в режиме простой замены.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] Алгоритм шифрования ГОСТ 28147-89 [Электронный ресурс]. – Режим доступа: <https://kaf401.rloc.ru/Criptfiles/gost28147/GOST28147.htm>. – Дата доступа: 02.09.2024.

[2] О шифровании ГОСТ 28147-89 [Электронный ресурс]. – Режим доступа: <https://habr.com/ru/articles/256843/>. – Дата доступа: 02.09.2024.

[3] Реализация алгоритма шифрования простой замены [Электронный ресурс]. – Режим доступа: <https://www.cyberforum.ru/thread1109400.html>. – Дата доступа: 02.09.2024.

ПРИЛОЖЕНИЕ А

(обязательное)

Листинг исходного кода

Листинг 1 – Исходный код

```
import struct
import re
from functools import partial

S_BOX = [
    [4, 10, 9, 2, 13, 8, 0, 14, 6, 11, 1, 12, 7, 15, 5, 3],
    [14, 11, 4, 12, 6, 13, 15, 10, 2, 3, 8, 1, 0, 7, 5, 9],
    [5, 8, 1, 13, 10, 3, 4, 2, 14, 15, 12, 7, 6, 0, 9, 11],
    [7, 13, 10, 1, 0, 8, 9, 15, 14, 4, 6, 12, 11, 2, 5, 3],
    [6, 12, 7, 1, 5, 15, 13, 8, 4, 10, 9, 14, 0, 3, 11, 2],
    [4, 11, 10, 0, 7, 2, 1, 13, 3, 6, 8, 5, 9, 12, 15, 14],
    [13, 11, 4, 1, 3, 15, 5, 9, 0, 10, 14, 7, 6, 8, 2, 12],
    [1, 15, 13, 0, 5, 7, 10, 4, 9, 2, 3, 14, 6, 11, 8, 12]
]

def cyclic_shift_left(value, shift):
    return ((value << shift) | (value >> (32 - shift))) & 0xFFFFFFFF

def F(block, key):
    temp = (block + key) & 0xFFFFFFFF
    for i in range(8):
        s_input = (temp >> (4 * i)) & 0xF
        s_output = S_BOX[i][s_input]
        temp = (temp & ~(0xF << (4 * i))) | (s_output << (4 * i))
    return cyclic_shift_left(temp, 11)

def encrypt_block(block, key):
    N1, N2 = block
    for i in range(32):
        round_key = key[i % 8]
        temp = F(N1, round_key) ^ N2
        N2, N1 = N1, temp
    return [N2, N1]

def decrypt_block(block, key):
    N1, N2 = block
    for i in range(31, -1, -1):
        round_key = key[i % 8]
        temp = F(N1, round_key) ^ N2
        N2, N1 = N1, temp
    return [N2, N1]

def read_file(filename):
    with open(filename, 'rb') as file:
        return file.read()

def write_file(filename, data):
    with open(filename, 'wb') as file:
        file.write(data)

def process_file(input_filename, output_filename, key, encrypt=True):
    data = read_file(input_filename)
    padding_size = (8 - (len(data) % 8)) % 8
    data += bytes([0] * padding_size)

    processed_data = bytearray()
```



```

for i in range(0, len(data), 8):
    block = struct.unpack('2I', data[i:i + 8])
    if encrypt:
        encrypted_block = encrypt_block(block, key)
    else:
        encrypted_block = decrypt_block(block, key)
    processed_data.extend(struct.pack('2I', *encrypted_block))

write_file(output_filename, processed_data)

def validate_input(message, pattern, converter):
    while True:
        user_input = input(message)
        if re.match(pattern, user_input):
            return converter(user_input)

def main():
    # 256 secret key
    key = [
        0xA56BABCD, 0xDEF01234, 0x789ABCDE, 0xFEDCBA98,
        0x01234567, 0x89ABCDEF, 0x12345678, 0x9ABCDEF0
    ]

    input_filename = "input.txt"
    encrypted_filename = "encrypted.bin"
    decrypted_filename = "decrypted.txt"

    options_pattern = r"^[1-2]$"
    option = validate_input("Choose option:\n1 - encrypt file\n2 - decrypt
file\n", options_pattern, int)

    if option == 1:
        process_file(input_filename, encrypted_filename, key, True)
        print(f"Encrypted file: {encrypted_filename}")
    elif option == 2:
        process_file(encrypted_filename, decrypted_filename, key, False)
        print(f"Decrypted file: {decrypted_filename}")

if __name__ == "__main__":
    main()

```